

Proxy, Reflect & Metaprogramming — Senior Developer Level

JavaScript-এর সাধারণ programming-এ আমরা object-এর property পড়ি, লিখি, function call করি, class ব্যবহার করি।

যেমন:

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
console.log(user.name);  
  
user.age = 32;
```

এখানে JavaScript আমাদের object-এর সাথে সরাসরি কাজ করার সুযোগ দিচ্ছে।

কিন্তু JavaScript-এর আরও একটি powerful capability আছে:

Object-এর সাধারণ operation-গুলোর behavior-এর মাঝখানে আমরা custom logic বসাতে পারি।

এখানেই আসে:

Proxy
Reflect
Metaprogramming
Decorators
Well-known Symbols

এই পুরো group-এর মূল ধারণা হলো:

Normal JavaScript

Code
↓
Object operation
↓
Result

Metaprogramming

Code

↓

Proxy / Protocol / Decorator

↓

Intercept / Modify behavior

↓

Result

1. Metaprogramming কী?

Metaprogramming হলো এমন programming যেখানে program নিজের behavior, structure বা execution-এর কিছু অংশকে inspect, modify বা control করতে পারে।

সহজভাবে:

Program-এর behavior নিয়ে programmatically কাজ করা।

উদাহরণ:

```
const user = {  
  name: "Sharlar"  
};  
  
user.name;
```

সাধারণ programming।

কিন্তু যদি আমরা চাই:

```
যখনই কেউ user.name পড়বে  
↓  
আমাদের custom function চলেবে  
↓  
তারপর আসল value return হবে
```

তাহলে সেটা metaprogramming-এর একটি উদাহরণ।

JavaScript-এ এর জন্য গুরুত্বপূর্ণ tools:

Proxy
Reflect
Symbols
Decorators
Property descriptors
Prototype manipulation

2. Proxy কী?

Proxy হলো এমন একটি object যা অন্য একটি object-এর operation-কে intercept বা customize করতে পারে।

Syntax:

```
const proxy = new Proxy(target, handler);
```

এখানে:

target
↓
Original object

handler
↓
কোন operation কীভাবে intercept হবে

proxy
↓
Target-এর controlled interface

উদাহরণ:

```
const user = {  
  name: "Sharir"  
};  
  
const proxy = new Proxy(user, {  
  get(target, property) {  
    console.log("Reading:", property);  
  
    return target[property];  
  }  
});
```

```
}  
});  
  
console.log(proxy.name);
```

Output-এর ধারণা:

```
Reading: name  
Shariar
```

এখানে:

```
proxy.name
```

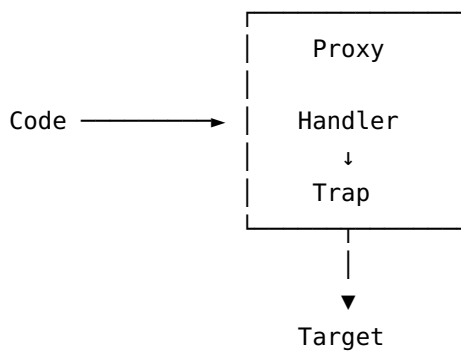
সরাসরি target-এর property access করেনি।

প্রথমে:

```
proxy  
↓  
get trap  
↓  
target.name
```

3. Proxy-এর মূল Mental Model

Proxy-কে ভাবতে পারো একটি **interceptor** হিসেবে।



অর্থাৎ:

```
Operation
↓
Proxy
↓
Trap
↓
Custom logic
↓
Target / custom result
```

4. Proxy কেন দরকার?

Proxy ব্যবহার করে অনেক ধরনের **behavior** তৈরি করা যায়:

- Validation
- Access control
- Logging
- Lazy loading
- Virtual properties
- API abstraction
- Reactive systems
- Dependency tracking
- Method interception
- Read-only objects
- Runtime contracts

Vue-এর মতো reactive system-এ Proxy-based reactivity-এর concept খুব গুরুত্বপূর্ণ।

5. Proxy-এর Trap কী?

Proxy-এর **trap** হলো handler-এর এমন method যা নির্দিষ্ট object operation intercept করে।

উদাহরণ:

```
{
  get() {},
  set() {},
  has() {}
}
```

এগুলোকে বলা হয় traps।

তোমার roadmap-এর প্রধান traps:

```
get
set
has
deleteProperty
apply
construct
```

এর বাইরেও Proxy-তে আরও traps আছে, যেমন:

```
getOwnPropertyDescriptor
defineProperty
ownKeys
getPrototypeOf
setPrototypeOf
isExtensible
preventExtensions
```

এগুলোও related হওয়ায় বুঝে রাখা দরকার।

6. `get` Trap

`get` trap property read intercept করে।

যেমন:

```
const user = {
  name: "Sharlar"
};

const proxy = new Proxy(user, {
  get(target, property) {
    console.log(`Reading ${String(property)}`);

    return target[property];
  }
});

console.log(proxy.name);
```

Flow:

```
proxy.name  
  ↓  
get trap  
  ↓  
target.name  
  ↓  
"Shariar"
```

7. `get` Trap-এর Third Argument

`get` trap-এর signature:

```
get(target, property, receiver)
```

যেমন:

```
const proxy = new Proxy(user, {  
  get(target, property, receiver) {  
    return Reflect.get(target, property, receiver);  
  }  
});
```

এখানে `receiver` গুরুত্বপূর্ণ বিশেষ করে inheritance এবং getter-এর ক্ষেত্রে।

8. `receiver` কেন গুরুত্বপূর্ণ?

ধরো:

```
const person = {  
  firstName: "Shariar",  
  
  get name() {  
    return this.firstName;  
  }  
};
```

এখন Proxy দিয়ে access করলে:

```
const proxy = new Proxy(person, {  
  get(target, property, receiver) {  
    return Reflect.get(target, property, receiver);  
  }  
});
```

এখানে getter-এর:

this

সঠিকভাবে proxy-এর দিকে নির্দেশ করতে পারে।

এটাই **receiver** ব্যবহারের একটি গুরুত্বপূর্ণ কারণ।

9. set Trap

set trap property assignment intercept করে।

```
const user = {  
  age: 25  
};  
  
const proxy = new Proxy(user, {  
  set(target, property, value) {  
    console.log(  
      `Setting ${String(property)} = ${value}`  
    );  
  
    target[property] = value;  
  
    return true;  
  }  
});  
  
proxy.age = 30;
```

Flow:

```
proxy.age = 30
  ↓
  set trap
  ↓
target.age = 30
```

10. `set` Trap-এর Return Value

`set` trap সাধারণত boolean return করে:

```
return true;
```

এর অর্থ:

Assignment সফল হয়েছে।

যদি:

```
return false;
```

হয়, তাহলে strict mode-এ assignment failure error তৈরি করতে পারে।

তাই `set` trap লিখলে return value সম্পর্কে সচেতন থাকতে হবে।

11. Validation দিয়ে `set`

Proxy-এর খুব practical use:

```
const user = {
  age: 20
};

const proxy = new Proxy(user, {
  set(target, property, value) {
    if (property === "age" && value < 0) {
      throw new Error("Age cannot be negative");
    }
  }
});
```

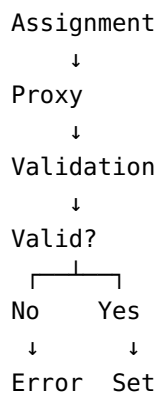
```
    return Reflect.set(target, property, value);
  }
});
```

এখন:

```
proxy.age = -10;
```

error হবে।

এখানে:



12. `has` Trap

`has` trap সাধারণত `in` operator intercept করে।

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {
  has(target, property) {
    console.log("Checking:", property);

    return property in target;
  }
});
```

```
console.log("name" in proxy);
```

Flow:

```
"name" in proxy
  ↓
  has trap
    ↓
property exists?
```

13. deleteProperty Trap

এটি:

```
delete proxy.property
```

intercept করে।

```
const user = {
  name: "Sharlar",
  age: 31
};

const proxy = new Proxy(user, {
  deleteProperty(target, property) {
    console.log("Deleting:", property);

    return Reflect.deleteProperty(target, property);
  }
});

delete proxy.age;
```

Flow:

```
delete proxy.age
  ↓
deleteProperty trap
  ↓
```

```
Reflect.deleteProperty()  
  ↓  
property deleted
```

14. `apply` Trap

এটি object property-এর জন্য নয়।

`apply` trap **function call** intercept করে।

যেহেতু JavaScript-এ function-ও object:

```
function add(a, b) {  
  return a + b;  
}
```

আমরা function-এর Proxy বানাতে পারি:

```
const proxy = new Proxy(add, {  
  apply(target, thisArg, argumentsList) {  
    console.log("Function called");  
  
    return Reflect.apply(  
      target,  
      thisArg,  
      argumentsList  
    );  
  }  
});  
  
proxy(10, 20);
```

Flow:

```
proxy(10, 20)  
  ↓  
  apply trap  
    ↓  
  Reflect.apply()
```

↓
add(10, 20)

15. apply Trap দিয়ে Logging

এটি খুব useful pattern:

```
function multiply(a, b) {  
  return a * b;  
}  
  
const logged = new Proxy(multiply, {  
  apply(target, thisArg, args) {  
    console.log("Arguments:", args);  
  
    const result = Reflect.apply(  
      target,  
      thisArg,  
      args  
    );  
  
    console.log("Result:", result);  
  
    return result;  
  }  
});
```

এখন:

logged(5, 4);

function-এর behavior-এর চারপাশে logging বসে গেল।

16. construct Trap

new operator intercept করতে construct trap ব্যবহৃত হয়।

```
class User {  
  constructor(name) {
```

```

        this.name = name;
    }
}

const ProxyUser = new Proxy(User, {
  construct(target, args, newTarget) {
    console.log("Creating User");

    return Reflect.construct(
      target,
      args,
      newTarget
    );
  }
});

const user = new ProxyUser("Sharir");

```

Flow:

```

new ProxyUser(...)
  ↓
construct trap
  ↓
Reflect.construct()
  ↓
User constructor
  ↓
instance

```

17. `construct` Trap কোথায় Useful?

ব্যবহার হতে পারে:

- Constructor logging
- Dependency injection
- Instance instrumentation
- Object creation monitoring
- Custom construction behavior

তবে normal application code-এ এর প্রয়োজন তুলনামূলক কম।

18. Other Important Proxy Traps

Roadmap-এ ৬টি trap থাকলেও পুরো Proxy system বুঝতে এগুলোও জানা দরকার।

```
get
set
has
deleteProperty
apply
construct

getOwnPropertyDescriptor
defineProperty
ownKeys
getPrototypeOf
setPrototypeOf
isExtensible
preventExtensions
```

এগুলো object-এর আরও low-level operations intercept করতে পারে।

19. ownKeys

যখন:

```
Object.keys(proxy)
```

বা related own-property enumeration হয়, `ownKeys` trap relevant হতে পারে।

```
const proxy = new Proxy(
  { name: "Sharlar", age: 31 },
  {
    ownKeys(target) {
      return ["name"];
    }
  }
);
```

এখানে object-এর property enumeration behavior customize করা হচ্ছে।

20. Proxy Invariants

এটা **Senior-level** গুরুত্বপূর্ণ topic ।

Proxy সবকিছু ইচ্ছামতো fake করতে পারে না ।

JavaScript কিছু **invariants** enforce করে ।

বিশেষ করে non-configurable বা non-writable property-এর ক্ষেত্রে Proxy handler এমন result দিতে পারে না যা language rules-এর সাথে সাংঘর্ষিক ।

উদাহরণ:

```
const target = {};  
  
Object.defineProperty(target, "x", {  
  value: 10,  
  writable: false,  
  configurable: false  
});
```

এখন Proxy-এর `get` trap দিয়ে:

```
get() {  
  return 20;  
}
```

এরকম incompatible behavior করার চেষ্টা error তৈরি করতে পারে ।

Mental model:

```
Proxy  
↓  
Custom behavior  
↓  
JavaScript invariants  
↓  
Must remain valid
```

অর্থাৎ:

Proxy powerful, কিন্তু language semantics ভেঙে ফেলার permission দেয় না ।

21. Reflect কী?

Reflect হলো JavaScript-এর built-in object, যেটি অনেক fundamental object operation-এর standard API দেয়।

যেমন:

```
Reflect.get()
Reflect.set()
Reflect.has()
Reflect.deleteProperty()
Reflect.apply()
Reflect.construct()
```

এগুলো Proxy traps-এর সাথে খুব naturally match করে।

22. Proxy + Reflect

একটি সাধারণ Proxy:

```
const proxy = new Proxy(target, {
  get(target, property, receiver) {
    return Reflect.get(
      target,
      property,
      receiver
    );
  }
});
```

এখানে Proxy সিদ্ধান্ত নিচ্ছে:

"আমি operation intercept করব।"

আর Reflect বলছে:

"এখন standard JavaScript semantics অনুযায়ী operation-টা execute করো।"

23. কেন Reflect ব্যবহার করব?

ধরো:

```
get(target, property) {  
  return target[property];  
}
```

অনেক simple ক্ষেত্রে এটি কাজ করবে।

কিন্তু:

```
get(target, property, receiver) {  
  return Reflect.get(  
    target,  
    property,  
    receiver  
  );  
}
```

আরও robust কারণ এটি standard internal operation-এর semantics-এর সাথে বেশি সরাসরি aligned।

বিশেষ করে:

- Getter
- Prototype
- `this`
- Receiver behavior

এসব ক্ষেত্রে পার্থক্য গুরুত্বপূর্ণ হতে পারে।

24. Proxy Trap → Reflect Mapping

Proxy Trap	Reflect API
<code>get</code>	<code>Reflect.get()</code>
<code>set</code>	<code>Reflect.set()</code>
<code>has</code>	<code>Reflect.has()</code>
<code>deleteProperty</code>	<code>Reflect.deleteProperty()</code>

Proxy Trap	Reflect API
<code>apply</code>	<code>Reflect.apply()</code>
<code>construct</code>	<code>Reflect.construct()</code>
<code>getOwnPropertyDescriptor</code>	<code>Reflect.getOwnPropertyDescriptor()</code>
<code>defineProperty</code>	<code>Reflect.defineProperty()</code>
<code>ownKeys</code>	<code>Reflect.ownKeys()</code>
<code>getPrototypeOf</code>	<code>Reflect.getPrototypeOf()</code>
<code>setPrototypeOf</code>	<code>Reflect.setPrototypeOf()</code>
<code>isExtensible</code>	<code>Reflect.isExtensible()</code>
<code>preventExtensions</code>	<code>Reflect.preventExtensions()</code>

এই mapping মনে রাখলে Proxy অনেক সহজ হয়ে যায়।

25. Reflect-এর আরেকটি সুবিধা — Boolean Result

কিছু traditional operations exception বা awkward behavior দিতে পারে।

Reflect-এর কিছু API success/failure-কে boolean result হিসেবে প্রকাশ করে।

উদাহরণ:

```
Reflect.defineProperty(
  target,
  "age",
  {
    value: 30
  }
);
```

এটি success হলে:

```
true
```

failure হলে:

false

এতে metaprogramming code অনেক predictable হতে পারে।

26. Proxy Handler-এ Reflect ব্যবহার করার Pattern

একটি clean pattern:

```
const proxy = new Proxy(target, {
  get(target, property, receiver) {
    // custom logic

    return Reflect.get(
      target,
      property,
      receiver
    );
  },

  set(target, property, value, receiver) {
    // validation / logging

    return Reflect.set(
      target,
      property,
      value,
      receiver
    );
  },

  has(target, property) {
    return Reflect.has(target, property);
  },

  deleteProperty(target, property) {
    return Reflect.deleteProperty(
      target,
      property
    );
  }
});
```

Mental model:

```
Intercept
  ↓
Custom logic
  ↓
Reflect
  ↓
Default semantics
```

27. Revocable Proxy

কখনো কখনো তুমি এমন Proxy চাইতে পারো যেটাকে পরে permanently disable করা যাবে।

এর জন্য:

```
Proxy.revocable()
```

ব্যবহার করা হয়।

```
const { proxy, revoke } = Proxy.revocable(
  target,
  handler
);
```

এখন:

```
proxy.name;
```

কাজ করবে।

কিন্তু:

```
revoke();
```

করার পর Proxy আর ব্যবহার করা যাবে না।

28. Revocable Proxy Example

```
const user = {
  name: "Shariar"
};

const { proxy, revoke } = Proxy.revocable(
  user,
  {
    get(target, property, receiver) {
      return Reflect.get(
        target,
        property,
        receiver
      );
    }
  }
);

console.log(proxy.name);

revoke();

console.log(proxy.name);
```

শেষের access error তৈরি করবে।

29. Revocable Proxy-এর Use Case

এটি useful হতে পারে:

- Temporary access
- Capability-based designs
- Resource lifetime control
- Plugin systems
- Security-sensitive wrappers
- Temporary object access

Mental model:

```
Access granted
  ↓
Proxy active
```

```
↓  
revoke()  
↓  
Access permanently disabled
```

30. Proxy vs Revocable Proxy

সাধারণ Proxy:

```
Proxy  
↓  
Remain usable
```

Revocable Proxy:

```
Proxy  
↓  
Active  
↓  
revoke()  
↓  
Dead
```

31. Proxy-এর একটি গুরুত্বপূর্ণ Limitation — Private Fields

JavaScript class-এর private field:

```
class User {  
  #secret = 123;  
  
  getSecret() {  
    return this.#secret;  
  }  
}
```

এখানে Proxy নিয়ে একটি subtle সমস্যা দেখা দিতে পারে।

যদি method call-এর `this` Proxy হয়ে যায়:

```
proxy
↓
method
↓
this = proxy
↓
#secret access
```

private field-এর brand check fail করতে পারে, কারণ private field target instance-এর সাথে tied থাকে।

এটি দেখায়:

Proxy সব internal language mechanism-এর transparent wrapper নয়।

এটি advanced Proxy usage-এ খুব গুরুত্বপূর্ণ বিষয়।

32. Decorators কী?

Decorator হলো এমন syntax/pattern যা class, method, field ইত্যাদির definition-এর সাথে metadata বা behavior modification-এর সুযোগ দেয়।

উদাহরণ:

```
@logged
class User {
  ...
}
```

অথবা method-level:

```
class User {
  @logged
  save() {
    ...
  }
}
```

তোমার roadmap-এ:

```
@logged
@memoize
@readonly
```

এর মতো examples আছে।

33. TC39 Decorators

JavaScript decorators নিয়ে অনেক বছর ধরে proposal development হয়েছে।

বর্তমান standardized decorators design-কে বুঝতে হলে পুরনো experimental decorator syntax-এর সাথে গুলিয়ে ফেলা যাবে না।

গুরুত্বপূর্ণ:

Decorator হলো language-level metaprogramming mechanism।

এটি Proxy-এর মতো runtime object operation interception-এর একই জিনিস নয়।

34. Decorator vs Proxy

Proxy:

```
Object operation
  ↓
intercept
```

Decorator:

```
Class / method / field definition
  ↓
decorate / modify / initialize
```

Mental model:

```
Proxy
→ runtime behavior interception
```

Decorator
→ definition-time/class-element customization

35. @logged Concept

ধরো:

```
class User {  
  save() {  
    return "saved";  
  }  
}
```

আমরা চাই:

```
save() call  
  ↓  
log  
  ↓  
original save()
```

Decorator conceptually এই behavior attach করতে পারে।

একটি conceptual implementation:

```
function logged(original, context) {  
  return function (...args) {  
    console.log(  
      `Calling ${String(context.name)}`  
    );  
  
    return original.apply(this, args);  
  };  
}
```

তারপর method-এ decorator ব্যবহার করা যায়।

এখানে গুরুত্বপূর্ণ বিষয় হলো:

```
original method
  ↓
decorator
  ↓
wrapped method
```

36. Decorator Context

Modern decorators design-এ decorator-কে context information দেওয়া হয়।

Conceptually:

```
function logged(value, context) {
  console.log(context.name);
  console.log(context.kind);

  return function (...args) {
    console.log("Called");

    return value.apply(this, args);
  };
}
```

`context` থেকে element সম্পর্কে metadata পাওয়া যায়।

যেমন:

```
name
kind
static
private
access
addInitializer
```

প্রয়োজন অনুযায়ী context-এর properties ব্যবহার করা যায়।

37. @memoize Concept

Memoization:

```
Input
↓
expensive calculation
↓
Result
```

একই input আবার এলে:

```
Input
↓
Cache hit
↓
Previous result
```

Decorator এই behavior automatically method-এর চারপাশে বসাতে পারে।

Conceptually:

```
method
↓
@memoize
↓
cache
↓
original method
```

38. @readonly Concept

একটি decorator class element-এর behavior এমনভাবে configure করতে পারে যাতে সেটি পরিবর্তন করা না যায়—তবে exact implementation এবং language semantics-এর ওপর নির্ভর করে এটি কীভাবে করা হবে।

এখানে গুরুত্বপূর্ণ lesson:

Decorator নিজে কোনো magical "readonly keyword" নয়; decorator ব্যবহার করে desired behavior তৈরি করা হয়।

39. Decorators vs Higher-Order Functions

JavaScript-এ আগে থেকেই function wrapping করা যায়:

```
const wrapped = logged(original);
```

Decorator syntax এটিকে declaration-এর কাছাকাছি নিয়ে আসে:

```
@logged  
method() {}
```

অর্থাৎ:

```
Decorator  
≈  
structured metaprogramming around class elements
```

40. Symbol.asyncIterator

এখন আসি JavaScript protocol-এর দিকে।

যদি কোনো object-কে:

```
for await (const item of object)
```

দিয়ে iterate করতে চাও, object-টি **async iterable** হতে পারে।

এর protocol:

```
Symbol.asyncIterator
```

41. Async Iterable

একটি async iterable object সাধারণত এমন একটি method expose করে:

```
[Symbol.asyncIterator]()
```

যেটি async iterator return করে।

Example:

```
const numbers = {
  async *[Symbol.asyncIterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};
```

এখন:

```
for await (const number of numbers) {
  console.log(number);
}
```

Output:

```
1
2
3
```

42. Async Iterator কী Return করে?

Async iterator-এর:

```
next()
```

একটি Promise return করে।

Conceptually:

```
iterator.next()
```

ফলাফল:

```
Promise<{
  value,
```

```
done  
}>
```

অর্থাৎ:

```
next()  
↓  
Promise  
↓  
{ value, done }
```

43. Async Iteration Protocol

পুরো structure:

```
Async Iterable  
|  
| Symbol.asyncIterator  
▼  
Async Iterator  
|  
| next()  
▼  
Promise  
|  
▼  
{ value, done }
```

এটাই `for await...of` বুঝতে মূল foundation।

44. `for await...of`

যেমন:

```
for await (const chunk of stream) {  
  await process(chunk);  
}
```

এখানে JavaScript:

```
iterator
  ↓
next()
  ↓
await Promise
  ↓
value
  ↓
next()
  ↓
...
```

এটি asynchronous data source-এর জন্য অত্যন্ত useful ।

45. Async Generator এবং `Symbol.asyncIterator`

Async generator:

```
async function* numbers() {
  yield 1;
  yield 2;
  yield 3;
}
```

এটি একটি async iterable object return করে।

```
for await (const number of numbers()) {
  console.log(number);
}
```

এখানে async generator নিজেই async iteration protocol implement করে।

46. কোথায় `Symbol.asyncIterator` Useful?

উদাহরণ:

```
Database cursor
  ↓
Async Iterator
```

↓
for await...of

অথবা:

Network stream
↓
Async Iterator
↓
for await...of

অথবা:

Paginated API
↓
Async Generator
↓
for await...of

এটি large/continuous data processing-এর সাথে খুব ভালোভাবে কাজ করে।

47. Async Iterable বনাম Async Iterator

এগুলো এক জিনিস নয়।

Async Iterable

যার:

[Symbol.asyncIterator]()

আছে।

Async Iterator

যার:

next()

আছে এবং Promise return করে।

Mental model:

```
Iterable
  ↓
gives Iterator

Async Iterable
  ↓
gives Async Iterator
```

48. Symbol.toPrimitive

এটি JavaScript-এর একটি অত্যন্ত interesting protocol।

যখন JavaScript কোনো object-কে primitive value-তে convert করতে চায়, তখন:

```
Symbol.toPrimitive
```

ব্যবহার করা যেতে পারে।

উদাহরণ:

```
const money = {
  amount: 100,

  [Symbol.toPrimitive](hint) {
    return this.amount;
  }
};
```

এখন:

```
console.log(money + 50);
```

object primitive conversion-এর মাধ্যমে:

```
money
  ↓
Symbol.toPrimitive
```

```
↓  
100  
↓  
100 + 50  
↓  
150
```

49. `Symbol.toPrimitive`-এর `hint`

Method-টি একটি `hint` পায়:

```
"number"  
"string"  
"default"
```

Example:

```
const value = {  
  [Symbol.toPrimitive](hint) {  
    console.log(hint);  
  
    return 10;  
  }  
};
```

তারপর `context` অনুযায়ী `hint` পরিবর্তিত হতে পারে।

50. `"number"` Hint

Numeric conversion-এর ক্ষেত্রে:

```
Number(value);
```

বা numeric operation-এর context-এ:

```
hint = "number"
```

আসতে পারে।

তখন `object` বলতে পারে:

```
return 100;
```

51. "string" Hint

String context:

```
String(value);
```

এর ক্ষেত্রে:

```
hint = "string"
```

ব্যবহার হতে পারে।

Example:

```
const user = {
  name: "Sharlar",

  [Symbol.toPrimitive](hint) {
    if (hint === "string") {
      return this.name;
    }

    return 1;
  }
};
```

52. "default" Hint

কিছু operator-এর ক্ষেত্রে:

```
hint = "default"
```

আসতে পারে।

যেমন:

```
value + anotherValue
```

এর মতো context-এ exact conversion behavior বুঝতে operator semantics-ও বিবেচনা করতে হয়।

তাই শুধু:

```
"+" → always number
```

এভাবে ভাবা ভাল।

53. `Symbol.toPrimitive` vs `valueOf()` vs `toString()`

Primitive conversion-এর ক্ষেত্রে JavaScript-এর fallback mechanisms আছে।

একটি simplified mental model:

```
Object conversion
  ↓
Symbol.toPrimitive?
  |
  yes
  ↓
use it

না হলে
  ↓
valueOf / toString
  ↓
primitive result
```

`Symbol.toPrimitive` থাকলে এটি conversion-এর প্রথম-class customization point হিসেবে কাজ করে।

54. Practical Money Example

ধরো:

```
class Money {  
  constructor(amount, currency) {  
    this.amount = amount;  
    this.currency = currency;  
  }  
  
  [Symbol.toPrimitive](hint) {  
    if (hint === "string") {  
      return `${this.amount} ${this.currency}`;  
    }  
  
    return this.amount;  
  }  
}
```

এখন:

```
const price = new Money(500, "BDT");
```

String context:

```
String(price);
```

ধারণাগত ফল:

```
500 BDT
```

Numeric context:

```
Number(price);
```

ধারণাগত ফল:

```
500
```

এটি object-এর primitive representation customize করছে।

55. Dangerous Side of Symbol.toPrimitive

এটি powerful হওয়ায় code confusing-ও করতে পারে।

যেমন:

```
console.log(price + 100);
```

একজন developer হয়তো ভাববে:

```
price is an object
```

কিন্তু object silently primitive conversion করে arithmetic করছে।

তাই domain-critical code-এ অতিরিক্ত magic conversion avoid করা ভালো।

56. Proxy + Reflect + Symbols

এই তিনটি concept একসাথে দেখলে JavaScript-এর metaprogramming system আরও পরিষ্কার হয়।

```
Proxy
↓
Object operation intercept

Reflect
↓
Standard object operation execute

Symbols
↓
Language protocols customize
```

উদাহরণ:

Proxy

- └─ get
- └─ set
- └─ apply
- └─ construct

Reflect

- └─ get
- └─ set
- └─ apply
- └─ construct

Symbols

- └─ Symbol.iterator
- └─ Symbol.asyncIterator
- └─ Symbol.toPrimitive

57. Proxy + Reflect Production Pattern

ধরো API client-এর জন্য automatic logging দরকার।

```
const api = {
  getUser(id) {
    return fetch(`/users/${id}`);
  },

  deleteUser(id) {
    return fetch(`/users/${id}`, {
      method: "DELETE"
    });
  }
};
```

Proxy দিয়ে:

```
const loggedApi = new Proxy(api, {
  get(target, property, receiver) {
    const original = Reflect.get(
      target,
      property,
      receiver
    );
```

```

    if (typeof original !== "function") {
      return original;
    }

    return function (...args) {
      console.log(
        `Calling ${String(property)}`
      );

      return Reflect.apply(
        original,
        this,
        args
      );
    };
  }
});

```

এখন:

```
loggedApi.getUser(10);
```

automatically logging করতে পারে।

58. Reactive System-এর সাথে Proxy

Proxy reactive systems তৈরিতে খুব useful।

Conceptually:

```

state.user.name
  ↓
Proxy get
  ↓
Track dependency

```

তারপর:

```

state.user.name = "New"
  ↓
Proxy set

```

↓
Trigger updates

Mental model:

```
Read
↓
get trap
↓
dependency tracking

Write
↓
set trap
↓
dependency trigger
```

এটি modern frontend frameworks-এর reactive architecture বোঝার জন্য useful ।

59. Validation Layer দিয়ে Proxy

ধরো:

```
const schema = {
  age: "number",
  name: "string"
};
```

Proxy:

```
assignment
↓
set trap
↓
validate
↓
valid?
├── No
└── Yes
```

```
↓      ↓  
Error  Reflect.set()
```

এভাবে runtime validation layer তৈরি করা যায়।

তবে production system-এ complex validation-এর জন্য dedicated validation library ব্যবহার করা অনেক সময় বেশি maintainable।

60. Proxy Performance Consideration

Proxy powerful, কিন্তু free নয়।

কারণ:

```
property access  
  ↓  
proxy layer  
  ↓  
trap  
  ↓  
custom logic  
  ↓  
target
```

অতিরিক্ত Proxy ব্যবহার performance ও debugging complexity বাড়াতে পারে।

বিশেষ করে hot path-এ:

```
for (...) {  
  proxy.value;  
}
```

হাজার/লাখ বার চললে overhead বিবেচনা করা উচিত।

Senior rule:

Proxy ব্যবহার করো যখন abstraction-এর value overhead-এর চেয়ে বেশি।

61. Proxy Debugging Complexity

Proxy ব্যবহার করলে code-এর apparent behavior এবং actual behavior আলাদা হতে পারে।

তুমি লিখছ:

```
user.name
```

কিন্তু ভিতরে:

```
get trap  
↓  
logging  
↓  
validation  
↓  
another proxy  
↓  
Reflect.get  
↓  
getter
```

চলতে পারে।

অতএব Proxy-heavy architecture-এ:

- Naming পরিষ্কার রাখো
- Trap ছোট রাখো
- Side effect সীমিত রাখো
- Documentation রাখো
- Debugging tools ব্যবহার করো

62. Proxy Trap-এ Infinite Recursion

এটি common mistake।

ভুল:

```
const proxy = new Proxy(target, {  
  get(target, property) {  
    return proxy[property];  
  }  
});
```

```
}  
});
```

কারণ:

```
proxy[property]  
↓  
get trap  
↓  
proxy[property]  
↓  
get trap  
↓  
∞
```

সঠিকভাবে target বা Reflect ব্যবহার করে:

```
get(target, property, receiver) {  
  return Reflect.get(  
    target,  
    property,  
    receiver  
  );  
}
```

63. Proxy-তে this সম্পর্কে সতর্কতা

Method wrapping করলে:

```
return function (...args) {  
  return Reflect.apply(  
    original,  
    this,  
    args  
  );  
};
```

এখানে this preserve করা গুরুত্বপূর্ণ।

ভুলভাবে:

```
original(...args);
```

করলে method-এর original receiver context হারাতে পারে।

64. Metaprogramming-এর তিনটি Level

এই পুরো group-কে তিন ভাগে ভাবতে পারো।

Level 1 — Interception

Proxy

Object operation intercept করে।

Level 2 — Standard Reflection

Reflect

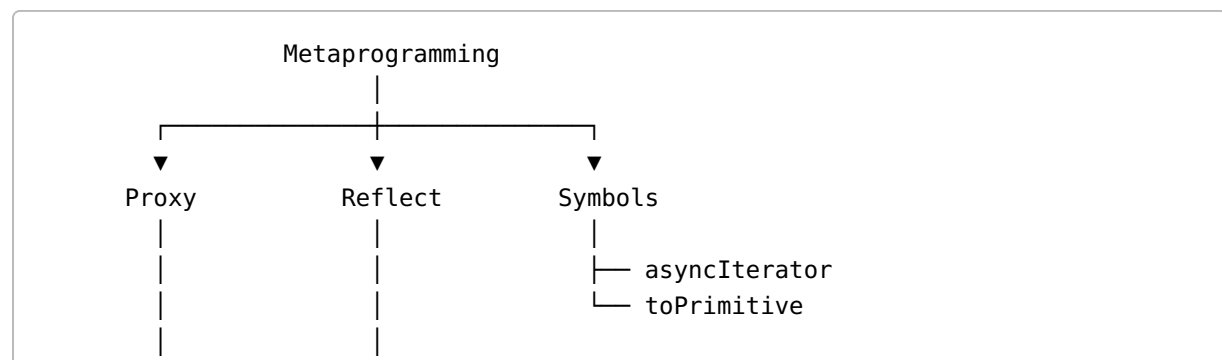
Language-level operations programmatically invoke করে।

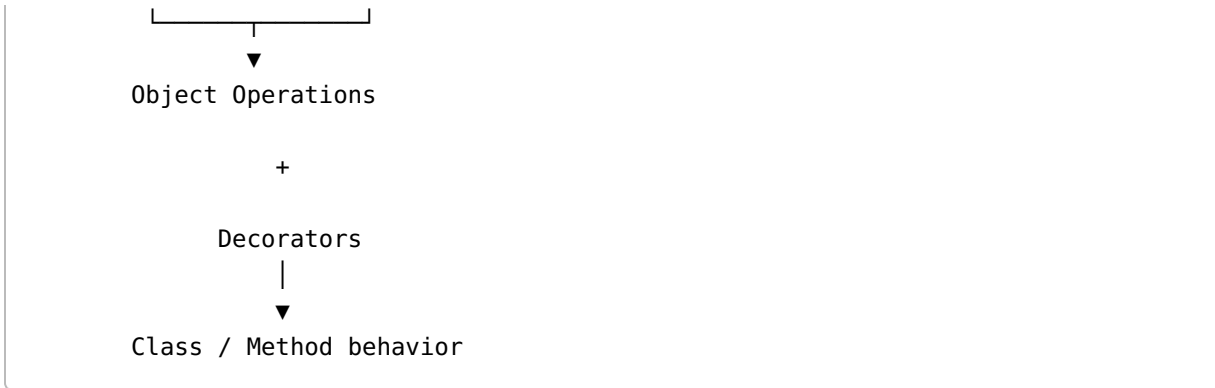
Level 3 — Protocol Customization

Symbols
Decorators

JavaScript-এর built-in protocols বা definitions customize করতে দেয়।

65. Complete Architecture





66. কখন **Proxy** ব্যবহার করবে?

Good use cases:

- Reactive state
- Validation
- Access control
- Logging/instrumentation
- Lazy properties
- Virtual objects
- API wrappers
- Plugin systems

67. কখন **Proxy** ব্যবহার না করাই ভালো?

যদি simple function দিয়ে কাজ হয়ে যায়:

```
validateUser(user);
```

তাহলে অকারণে:

```
new Proxy(...)
```

ব্যবহার করার দরকার নেই।

কারণ:

```
Simple abstraction
>
Hidden magic
```

যতক্ষণ না Proxy-এর capability সত্যিই প্রয়োজন হচ্ছে।

68. কখন **Reflect** ব্যবহার করবে?

Proxy trap-এর ভিতরে default behavior preserve করতে:

```
return Reflect.get(...);
```

অথবা:

```
return Reflect.set(...);
```

এটি খুব natural use case।

এছাড়া low-level object operations programmatically করতে Reflect নিজেও useful।

69. কখন **Decorator** ব্যবহার করবে?

যখন একই ধরনের class/method behavior বারবার লাগছে:

```
Logging
Authorization
Instrumentation
Registration
Validation
Caching
Initialization
```

তখন decorator clean abstraction দিতে পারে।

70. কখন `Symbol.asyncIterator` ব্যবহার করবে?

যখন তোমার data source asynchronous:

```
Database cursor  
API pagination  
Network stream  
File processing  
Message source
```

এবং তুমি চাই:

```
for await (const item of source) {  
    ...  
}
```

তখন async iteration protocol অত্যন্ত useful।

71. কখন `Symbol.toPrimitive` ব্যবহার করবে?

যখন কোনো domain object-এর primitive representation meaningful:

```
Money  
Duration  
Measurement  
Custom numeric-like value
```

তবে implicit conversion-এর কারণে code confusing হতে পারে, তাই carefully ব্যবহার করতে হবে।

72. Common Mistakes

Mistake 1

Proxy = validation tool ভাবা।

না।

Proxy অনেক broader:

interception + metaprogramming

Mistake 2

Proxy আর Reflect একই জিনিস ভাবা।

না।

Proxy
→ intercept

Reflect
→ perform standard operation

Mistake 3

সব Proxy trap-এ manually target access করা।

অনেক ক্ষেত্রে:

```
Reflect.get(...)  
Reflect.set(...)
```

ব্যবহার করা cleaner।

Mistake 4

Shared object-এর সব behavior Proxy দিয়ে magic করা।

এতে debugging কঠিন হয়ে যেতে পারে।

Mistake 5

`Symbol.asyncIterator` আর `Symbol.iterator` গুলিয়ে ফেলা।

`Symbol.iterator`
→ synchronous iteration

```
Symbol.asyncIterator  
→ asynchronous iteration
```

Mistake 6

`Symbol.toPrimitive` -কে শুধু `toString()` ভাব।

এটি broader primitive conversion protocol।

Mistake 7

Modern decorators আর পুরনো experimental decorator semantics এক করে দেখা।

Decorator ecosystem-এর version/configuration-এর উপর exact syntax/behavior নির্ভর করতে পারে। তাই project-এর actual JavaScript/TypeScript toolchain-এর semantics যাচাই করা গুরুত্বপূর্ণ।

73. Quick Revision

Proxy

```
Object operation  
  ↓  
Trap  
  ↓  
Custom behavior
```

Reflect

Standard object operation API

Common:

```
Reflect.get()  
Reflect.set()  
Reflect.has()
```

```
Reflect.apply()  
Reflect.construct()
```

Revocable Proxy

```
Proxy  
↓  
revoke()  
↓  
Access disabled
```

Decorators

```
Class / method / field  
↓  
Decorator  
↓  
Behavior / initialization / metadata customization
```

Symbol.asyncIterator

```
Async Iterable  
↓  
Symbol.asyncIterator  
↓  
Async Iterator  
↓  
next()  
↓  
Promise
```

Symbol.toPrimitive

```
Object  
↓
```

```
primitive conversion
↓
Symbol.toPrimitive(hint)
↓
primitive
```

Hints:

```
number
string
default
```

74. Interview Questions

Fundamentals

1. Proxy কী?
2. Proxy এবং normal object-এর মধ্যে পার্থক্য কী?
3. Proxy trap কী?
4. `get` এবং `set` trap কীভাবে কাজ করে?
5. `apply` এবং `construct` trap-এর পার্থক্য কী?

Reflect

1. Reflect কী?
2. Proxy-এর সাথে Reflect কেন এত বেশি ব্যবহার করা হয়?
3. `Reflect.get()` এবং `target[property]` -এর মধ্যে কী পার্থক্য হতে পারে?
4. `receiver` কী?
5. Proxy invariants কী?

Advanced Proxy

1. Revocable Proxy কী?
2. Proxy দিয়ে validation কীভাবে implement করবে?
3. Proxy দিয়ে logging কীভাবে implement করবে?
4. Proxy-তে infinite recursion কীভাবে হতে পারে?
5. Proxy এবং private class fields-এর interaction-এ কী সমস্যা হতে পারে?
6. Proxy-এর performance cost কেন হতে পারে?

Decorators

1. Decorator কী?
2. Decorator এবং Proxy-এর মূল পার্থক্য কী?
3. Method decorator কীভাবে method wrap করতে পারে?

4. Memoization decorator কীভাবে কাজ করতে পারে?

Symbols

1. `Symbol.iterator` এবং `Symbol.asyncIterator` -এর পার্থক্য কী?
2. Async iterable কী?
3. Async iterator-এর `next()` কী return করে?
4. `for await...of` কীভাবে async iteration protocol ব্যবহার করে?
5. `Symbol.toPrimitive` কী?
6. `number`, `string`, `default` hint কী?
7. `Symbol.toPrimitive`, `valueOf()` এবং `toString()` -এর সম্পর্ক কী?

75. Practice Tasks

Task 1 — Logging Proxy

একটি object বানাও:

```
const user = {  
  name: "Sharlar",  
  age: 31  
};
```

Proxy দিয়ে:

```
read → log  
write → log  
delete → log
```

করো।

Task 2 — Validation Proxy

Rules:

```
age → number  
name → string
```

Invalid value দিলে error throw করবে।

Task 3 — Readonly Proxy

এমন Proxy বানাও যেখানে:

```
proxy.name = "New";
```

করলে error হবে।

কিন্তু original target-এর behavior অক্ষত থাকবে।

Task 4 — Function Proxy

একটি function-এর Proxy বানিয়ে:

```
function call  
↓  
arguments log  
↓  
execution time  
↓  
result
```

measure করো।

Task 5 — Revocable Access

একটি temporary capability তৈরি করো:

```
create access  
↓  
use proxy  
↓  
revoke  
↓  
access denied
```

Task 6 — Async Iterable

একটি async iterable বানাও যেটি প্রতি 1 second পর:

```
1  
2  
3  
4  
5
```

দেবে।

তারপর:

```
for await...of
```

দিয়ে consume করো।

Task 7 — Symbol.toPrimitive

একটি Money class বানাও:

```
const price = new Money(500, "BDT");
```

যেখানে:

```
String(price)
```

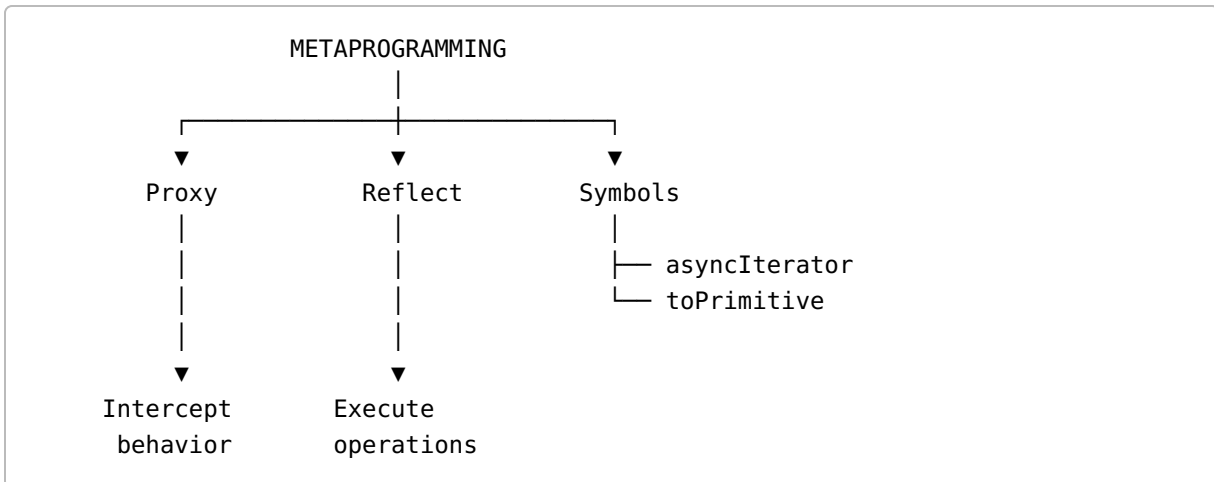
মানুষের পড়ার মতো value দেয় এবং:

```
Number(price)
```

numeric amount দেয়।

76. Final Senior Mental Model

এই পুরো group-টা মনে রাখার সবচেয়ে সহজ উপায়:



আর:

Proxy

↓

"What happens when someone
accesses/calls/modifies this?"

Reflect

↓

"Perform the normal operation
programmatically."

Symbol.asyncIterator

↓

"How should this object behave
during async iteration?"

Symbol.toPrimitive

↓

"How should this object become
a primitive value?"

Decorator

↓

"How should this class/method/
field be customized?"

সবশেষে একটি খুব important distinction:

Proxy

→ Runtime operation interception

Reflect

→ Standard reflective operations

Decorator

→ Class/element customization

Symbol protocols

→ JavaScript language protocols customization

এই distinction-গুলো পরিস্কার থাকলে JavaScript-এর metaprogramming আর "magic" মনে হবে না। তুমি বুঝতে পারবে কোন **mechanism** কোন স্তরে কাজ করছে, কেন ব্যবহার করা হচ্ছে, এবং কোথায় অতিরিক্ত **abstraction code**-কে বরং জটিল করে ফেলতে পারে।